

Frontend Developer — Step 3

LEARN JAVASCRIPT

Detailed Notes: Fundamentals → DOM → ES6+ → Asynchronous JavaScript

CareerCompass Learning Resource

Beginner-friendly notes with concepts, examples, execution flow, browser APIs, modern syntax, async programming and practical exercises.

Learning Map

JavaScript adds behavior and logic to webpages. It can read and change the DOM, respond to events, work with data, call APIs and coordinate asynchronous operations.

User action → Event → JavaScript logic → DOM/API/state update → Browser renders updated UI

1	JavaScript Fundamentals
2	Variables, Data Types & Operators
3	Conditions, Loops & Functions
4	Arrays & Objects
5	Scope, Hoisting & Execution
6	DOM Manipulation
7	Events & Forms
8	ES6+ Modern JavaScript
9	Modules & Error Handling
10	Asynchronous JavaScript
11	Promises, async/await & Fetch
12	Practical Project
13	Debugging & Best Practices
14	Practice + Interview Checklist

1. JavaScript Fundamentals

What is JavaScript?

JavaScript is a programming language widely used to make web pages interactive and dynamic. In the browser it can respond to user actions, modify the DOM, validate forms, manage UI state and communicate with servers.

First example

```
console.log("Hello, JavaScript!");

const name = "Manthan";
console.log(`Hello ${name}`);
```

Where JavaScript runs

Browser JavaScript runs inside a JavaScript engine. Modern browsers use engines such as V8, SpiderMonkey or JavaScriptCore. JavaScript can also run outside the browser, for example with Node.js.

Adding JavaScript to HTML

```
<script src="script.js" defer></script>
```

Why defer?

The defer attribute lets an external script download while HTML parsing continues and executes it after the document has been parsed, while preserving script order.

2. Variables, Data Types & Operators

Variables

```
let age = 20;
const name = "Manthan";
var oldStyle = true;
```

let vs const vs var

Use const when the binding should not be reassigned. Use let when reassignment is needed. var is function-scoped and has older semantics; prefer let/const in modern code.

Primitive values

Common primitive types are string, number, bigint, boolean, undefined, symbol and null. Objects, arrays and functions are non-primitive values.

Operators

```
const total = 10 + 5;
const isAdult = age >= 18;
const message = "Hello " + name;
const modern = `Age: ${age}`;
```

Equality

Prefer strict equality (===) and strict inequality (!==) when you want predictable comparisons without implicit type coercion.

Truthy and falsy

In conditions, values such as false, 0, -0, 0n, an empty string, null and undefined are falsy. Most other values are truthy.

3. Conditions, Loops & Functions

Conditionals

```
if (score >= 90) {
  console.log("Excellent");
} else if (score >= 40) {
  console.log("Pass");
} else {
  console.log("Try again");
}
```

Switch

```
switch (role) {
  case "admin":
    console.log("Full access");
    break;
  case "user":
    console.log("Standard access");
    break;
  default:
    console.log("Unknown role");
}
```

Loops

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}

for (const item of items) {
  console.log(item);
}
```

Functions

```
function add(a, b) {
  return a + b;
}

const result = add(10, 20);
```

Function concepts

Understand parameters, arguments, return values, local variables, default parameters and the difference between a function declaration and a function expression.

4. Arrays & Objects

Arrays

```
const skills = ["HTML", "CSS", "JavaScript"];
skills.push("React");
console.log(skills[0]);
```

Useful array methods

Learn push, pop, shift, unshift, slice, splice, includes, find, some, every, map, filter and reduce. Understand that map/filter/reduce return new results rather than simply changing the original array.

Objects

```
const user = {
  name: "Manthan",
  age: 20,
  role: "Frontend Developer"
};
```

```
console.log(user.name);
user.age = 21;
```

Destructuring

```
const { name, role } = user;
const [first, second] = skills;
```

JSON

JSON is a text data format commonly used when exchanging structured data between a browser and a server.

JSON.parse converts JSON text into a JavaScript value; JSON.stringify converts a JavaScript value into JSON text.

5. Scope, Hoisting & Execution

Scope

Scope determines where a variable can be accessed. JavaScript has global scope, function scope and block scope. let and const are block-scoped; var is function-scoped.

Example

```
let globalValue = "global";

function demo() {
  let localValue = "local";
  console.log(globalValue);
  console.log(localValue);
}
```

Lexical scope

A nested function can access variables from its surrounding scopes. This behavior is the foundation for closures.

Hoisting

JavaScript processes declarations before executing the associated code. Function declarations can be called before their position in source code. let/const declarations are hoisted but cannot be accessed before initialization because of the temporal dead zone.

Closures

A closure occurs when a function retains access to variables from its lexical environment even after the outer function has returned.

6. DOM Manipulation

What is the DOM?

The Document Object Model represents the HTML document as a tree of objects. JavaScript can use DOM APIs to find elements, read values, change content, change classes and create/remove nodes.

Selecting elements

```
const title = document.querySelector("h1");
const buttons = document.querySelectorAll(".btn");
```

Changing content

```
title.textContent = "Learn JavaScript";

const box = document.querySelector(".box");
box.classList.add("active");
```

Creating elements

```
const li = document.createElement("li");
li.textContent = "React";
document.querySelector("ul").append(li);
```

Attributes and styles

```
const img = document.querySelector("img");
img.setAttribute("alt", "Course illustration");
```

```
box.style.display = "none";
```

Best practice

Prefer classList and CSS classes for reusable styling instead of setting many inline styles from JavaScript.

7. Events & Forms

Events

An event represents something that happened, such as click, input, submit, change, keydown or mouseover.

Event listener

```
const button = document.querySelector("#save");
button.addEventListener("click", () => {
  console.log("Saved");
});
```

Event object

```
button.addEventListener("click", (event) => {
  console.log(event.target);
});
```

Form handling

```
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const email = emailInput.value.trim();
  console.log(email);
});
```

Event delegation

When many similar child elements exist, a listener on a parent can inspect event.target and handle interactions efficiently. This is especially useful for dynamically created lists.

8. ES6+ Modern JavaScript

let and const

Use block-scoped let and const instead of var for most modern code.

Arrow functions

```
const square = n => n * n;
const add = (a, b) => a + b;
```

Template literals

```
const user = "Manthan";
const message = `Welcome, ${user}!`;
```

Spread and rest

```
const a = [1, 2];
const b = [...a, 3, 4];

function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
```

Destructuring

```
const { name, age } = user;
const [first, second] = numbers;
```

Optional chaining and nullish coalescing

```
const city = user.address?.city;
const label = user.name ?? "Guest";
```

Classes

```
class User {
  constructor(name) {
    this.name = name;
  }
  greet() {
    return `Hello ${this.name}`;
  }
}
```

Modules

Use export and import to split larger programs into maintainable modules.

9. Modules & Error Handling

ES modules

```
// math.js
export function add(a, b) { return a + b; }

// app.js
import { add } from "./math.js";
```

HTML module script

```
<script type="module" src="app.js"></script>
```

try/catch

```
try {
  const data = JSON.parse(input);
  console.log(data);
} catch (error) {
  console.error("Invalid data", error);
}
```

Throwing errors

```
function divide(a, b) {
  if (b === 0) throw new Error("Cannot divide by zero");
  return a / b;
}
```

Good error handling

Catch errors where you can actually recover or provide useful feedback. Do not hide errors with empty catch blocks.

10. Asynchronous JavaScript

Why asynchronous code?

Network requests, timers and other operations may take time. JavaScript can start an operation and continue executing other work instead of blocking the entire UI while waiting.

Synchronous vs asynchronous

Synchronous code generally proceeds step by step. Asynchronous APIs let work complete later and notify your code through callbacks, promises or `async/await`.

Timer example

```
console.log("A");
setTimeout(() => console.log("B"), 1000);
console.log("C");

// Output begins: A, C, then B
```

Event loop mental model

A useful simplified model is: call stack executes current JavaScript; asynchronous APIs schedule completion; queued callbacks/promises are processed when the stack is ready. Promise reactions are handled through the microtask queue.

11. Promises, `async/await` & Fetch

Promise states

A Promise represents a future result. It can be pending, fulfilled or rejected.

Promise example

```
fetch("/api/careers")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

`async/await`

```
async function loadCareers() {
  try {
    const response = await fetch("/api/careers");
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);
    }
    const data = await response.json();
    return data;
  } catch (error) {
    console.error(error);
  }
}
```

Fetch flow

Request → server response → check `response.ok` → parse body → use data → handle errors.

Promise.all

```
const [users, careers] = await Promise.all([
  fetch("/api/users").then(r => r.json()),
  fetch("/api/careers").then(r => r.json())
]);
```

Important

`fetch()` rejects for some network failures, but an HTTP 404/500 response does not automatically reject. Check `response.ok` when appropriate.

12. Practical JavaScript Project

Project

Build a Career Search app using HTML/CSS/JavaScript. Load career data from a JSON file or API, display cards, implement search/filter, show a detail section and handle loading/error/empty states.

Suggested structure

```
fetch("/api/careers")
  .then(r => {
    if (!r.ok) throw new Error("Request failed");
    return r.json();
  })
  .then(careers => renderCareers(careers))
  .catch(error => showError(error));
```

DOM rendering idea

```
function renderCareers(careers) {
  const container = document.querySelector("#careers");
  container.innerHTML = careers.map(career => `
    <article class="career-card">
      <h2>${career.title}</h2>
      <p>${career.description}</p>
    </article>
  `).join("");
}
```

Project features

Search by title; filter by category; click a card for details; show loading state; show an error message when API fails; show a friendly empty state when no result matches.

13. Debugging & Best Practices

DevTools

Use Console for errors/logs, Sources for stepping through code, Network for API requests/responses, and Elements for DOM inspection.

Common problems

If an element is null, check selector timing and spelling. If fetch data is missing, inspect the Network tab and response status. If a click works only sometimes, inspect event delegation and dynamically created elements.

Best practices

Use `const` by default; use `let` when reassignment is needed; prefer strict equality; keep functions small; use meaningful names; avoid global variables; validate external data; handle async errors; separate DOM, data and business logic where practical.

14. Practice + Interview Checklist

Practice tasks

Build a calculator; to-do list; form validator; search/filter UI; dynamic table; weather/API card; quiz app; notes app; career search app.

Interview questions

What is JavaScript? let vs const vs var? Primitive vs reference values? == vs ===? What is scope? What is hoisting? What is a closure? What is the DOM? Event bubbling? Event delegation? What is a Promise? What is the event loop? async/await vs then/catch? What does fetch return? Why check response.ok? What are ES modules?

Mastery checklist

I can use variables and data types; conditions and loops; functions; arrays and objects; destructuring; DOM selection/manipulation; events and forms; modern ES6+ syntax; modules; error handling; promises; async/await; fetch; DevTools; and build a real interactive project.

JavaScript Execution & Async Quick Reference

A simplified browser model: JavaScript executes on the call stack. Browser APIs can perform timers, networking and other work. When work completes, callbacks/tasks or promise reactions are queued. The event loop coordinates when queued work can run.

For CareerCompass frontend work, the most important practical chain is: user action → event handler → validation/state logic → DOM update or fetch → response → render UI.

Reference: MDN Web Docs JavaScript documentation is recommended for standards-based reference and deeper API/property details.